Programming Fundamentals 2

Topic 07 - Tutorial Sheet 7.1

Manual Testing to JUnit

Complete the tasks in order. Start by reasoning from the specification, then write manual test data, then implement the method, and finally translate the same tests into JUnit.

Method specification

You are asked to develop a method `calculateGrade(int mark)` that returns a grade using the rules below:

- `0-39` -> `"Fail"`
- `40-54` -> `"Pass"`
- `55-69` -> `"Merit"`
- `70-100` -> `"Distinction"`

If the input is invalid, the method should return exactly:

```
"Invalid Input - number should be between 0 and 100 inclusive"
```

Exercise 1 - Start with manual test data

Without writing any Java yet, invent at least six manual test cases for the method from the specification.

For each case, record the input, the expected result, and why that case is worth keeping.

Test case	mark	Expected result / behaviour	Reason
1			
2			
3			
4			
5			
6			

Exercise 2 - Check whether you have covered the main possibilities

Use the categories below to review your test data. Add a concrete mark and the expected result for each category.

A strong test set should cover ordinary cases, boundaries, and invalid input.

Category	Example mark	Expected result
Typical Fail		
Typical Pass		
Typical Merit		
Typical Distinction		
Lowest valid mark		
Highest Fail		
Lowest Pass		
Highest Pass		
Lowest Merit		
Highest Merit		
Lowest Distinction		
Highest valid mark		
Below the valid range		
Above the valid range		

Exercise 3 - Reduce your ideas to a final test set

Choose a final set of test values that gives good coverage without repeating the same idea too many times.

Keep the cases that tell you something important about the specification.

Keep?	mark	Expected result	Why this case earns a place
[ ]			
[ ]			
[ ]			
[ ]			
[ ]			
[ ]			
[ ]			
[ ]			

Exercise 4 - Write the method

Now implement `calculateGrade(int mark)` so that it matches the specification exactly.

Write the method in your IDE. Use the space below for planning, pseudocode, or a first draft.

```
public static String calculateGrade(int mark) {



}
```

Exercise 5 - Turn the same test data into JUnit tests

Do not invent a new set of tests here. Reuse the manual test data you designed earlier.

Your JUnit tests should reflect the specification and the test values you chose.

Manual test value	Expected result	Name for the JUnit test method

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

public class GradeCalculatorTest {

    @Test
    public void testExample() {
        assertEquals("", GradeCalculator.calculateGrade(0));
    }
}
```

Exercise 6 - Boundary-value focus

Complete the table below using the specification. These values sit on, or just outside, the grade boundaries.

This is where many bugs like to live. They are the houseplants of the testing world: always at the edge.

mark	Expected result
-1	
0	
39	
40	
54	
55	
69	
70	
100	
101	

Final check before submission

Checkpoint	Tick
My manual test data includes ordinary cases and invalid input.	[ ]
My final test set includes important boundary values.	[ ]
My method returns the exact error string shown in the specification.	[ ]
My JUnit tests use the same values I chose during manual testing.	[ ]
My method passes the tests I wrote.	[ ]

Reflection

1. Which test values were the most important in this exercise, and why?

2. Which bugs would be most likely if you forgot to test the boundaries?